Análisis y Diseño de Algoritmos I

Valeria Bengolea
Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales
Universidad Nacional de Río Cuarto

Técnicas de Diseño: Divide & Conquer

Notas basadas en el material preparado por el Dr. Nazareno Aguirre

Estrategias de Diseño de Algoritmos

Las estrategias de diseño de algoritmos nos brindan herramientas para atacar el problema de construir soluciones algorítmicas para problemas.

Hemos visto una de ellas, Fuerza Bruta, que es muy general pero también tiene limitaciones.

Divide & Conquer

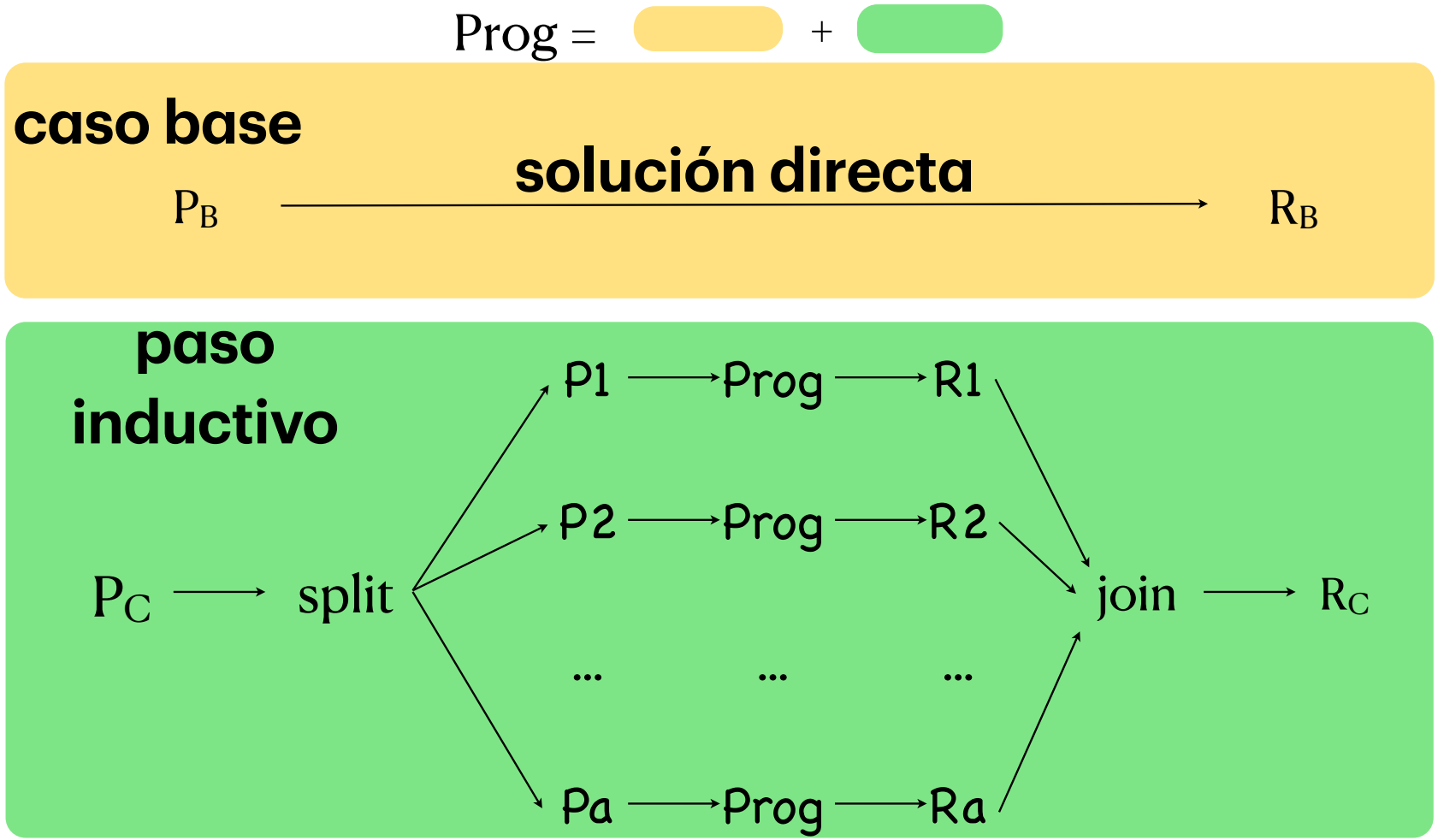
Veremos ahora una segunda estrategia, **Divide & Conquer**, que corresponde a un mecanismo de resolución de problemas muy general (admite su aplicación en una amplia cantidad de casos) y suele ofrecer soluciones eficientes.

Divide & Conquer

Una forma general de describir Divide & Conquer, es la siguiente

- Si el problema puede resolverse directamente, se resuelve sino:
- Un problema se divide en varios subproblemas del mismo tipo, idealmente del mismo tamaño
- Se resuelve cada subproblema , en general de manera recursiva.
- Las soluciones de cada subproblema se combinan para conseguir una solución al problema original.

Esquema General de la Estrategia Divide & Conquer



Divide & Conquer

Una forma general de describir Divide & Conquer, es la siguiente:

Una solución Divide & Conquer para un problema P dado consiste en:

- identificar un conjunto de instancias del problema P elementales, para las cuales la solución es inmediata.
- para aquellas instancias de P que no sean elementales, encontrar una forma de resolverlas en términos de soluciones para instancias del problema P "de menor tamaño", a través de un patrón **dividir - conquistar - combinar**.

En esta descripción de Divide & Conquer aparecen algunos conceptos importantes, como el concepto de "problema elemental", y de "tamaño del problema".

Divide & Conquer y Recursión

Existe una clara correspondencia entre la estrategia de diseño Divide & Conquer y recursión:

- los problemas elementales corresponden a los casos base
- resolución de problemas de menor tamaño y su combinación corresponden al paso inductivo

La reducción a problemas de menor tamaño debe garantizar que sucesivas reducciones eventualmente alcanzan los casos base

Ejemplo: Ordenamiento

Veamos como ejemplo el problema de ordenar una secuencia de elementos, y un par de soluciones Divide & Conquer para el problema.

Mergesort es un ejemplo de una aplicación de la técnica Divide & Conquer:

Ordena un arreglo A[0...n-1] dividiéndolo en 2 mitades A[0.. $\lfloor n/2 \rfloor - 1$] and A[$\lfloor n/2 \rfloor$ ..n – 1], ordenando (recursivamente) las dos mitades, y finalmente mezclando las dos mitades ordenadas en un arreglo ordenado.

Ejemplo: Ordenamiento

Mergesort:

caso base: secuencias de tamaño menor o igual a 1.

Solución: retornar la misma secuencia (ya está ordenada)

paso inductivo (para secuencias de tamaño mayor a 1):

Split: separar la secuencia en dos partes de (aproximadamente) el mismo tamaño

Split.xs = (take.#xs/2.xs, drop.#xs/2.xs)

Join: Realizar la "mezcla ordenada" de las dos secuencias (ya ordenadas)

Join.xs.ys = merge.xs.ys

Ejemplo: Ordenamiento

Mergesort:

ALGORITHM *Mergesort*(A[0..n − 1])
//Sorts array *A*[0..n − 1] by recursive mergesort
//Input: An array *A*[0..n − 1] of orderable elements
//Output: Array *A*[0..n − 1] sorted in nondecreasing order
if n > 1

copy A[0..⌊n/2⌋ − 1] to B[0..⌊n/2⌋ − 1]
copy A[⌊n/2⌋..n − 1] to C[0..⌈n/2⌉ − 1]

Split

Mergesort(B[0..⌊n/2⌋ − 1])
Mergesort(C[0..⌈n/2⌉ − 1])

Merge(*B*, *C*, *A*)

Join

Ejemplo: Ordenamiento

ALGORITHM *Mergesort*(A[0..n − 1])
if n > 1

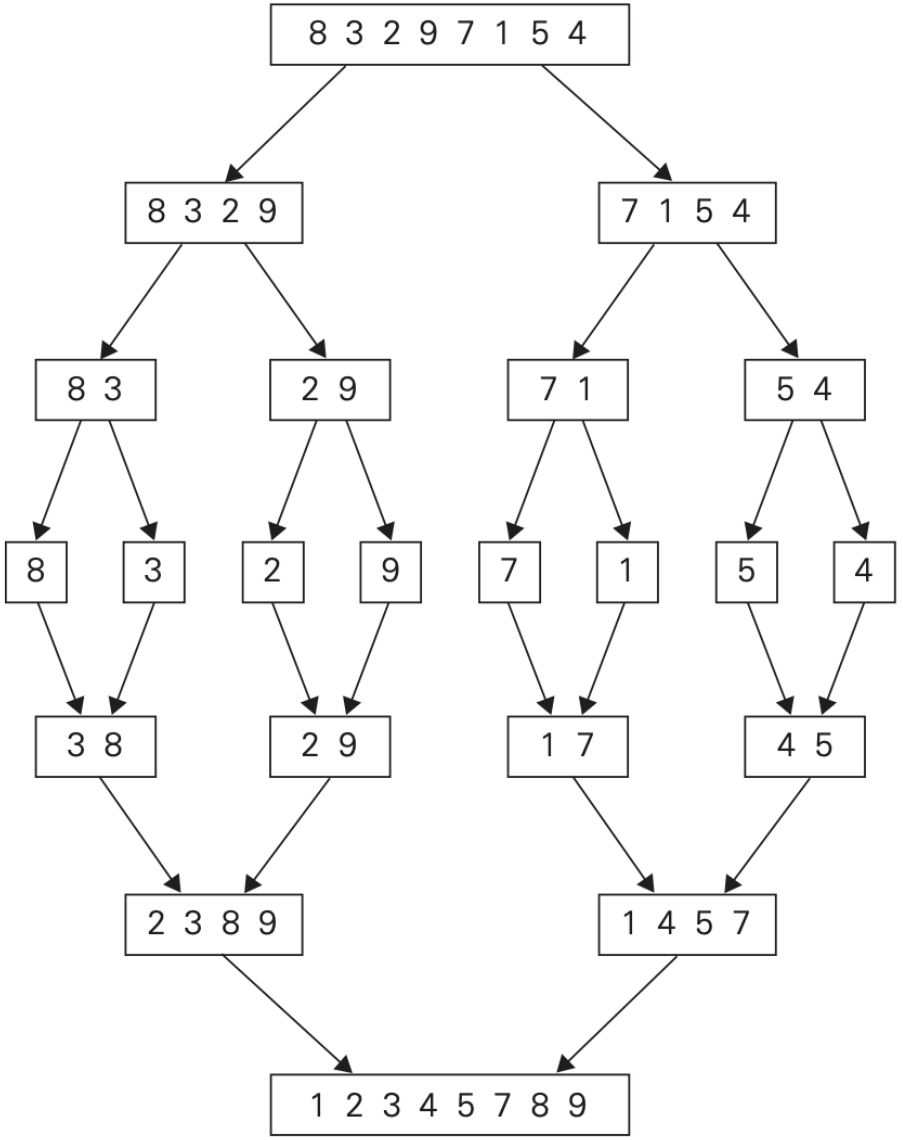
copy A[0.. ⌊n/2⌋ − 1] to B[0.. ⌊n/2⌋ − 1]

copy A[⌊n/2⌋..n − 1] to C[0.. ⌈n/2⌉ − 1]

Mergesort(B[0.. ⌊n/2⌋ − 1])

Mergesort(C[0.. ⌈n/2⌉ − 1])

Merge(*B, C, A*)



Ejemplo de Análisis de Tiempo de Ejecución

El análisis de soluciones Divide & Conquer recursivas demanda la resolución de ecuaciones de recurrencia. Consideremos el caso de MergeSort sobre un lenguaje imperativo:

ALGORITHM *Mergesort*(A[0..n − 1])
if n > 1

copy A[0.. ⌊n/2⌋ − 1] to B[0.. ⌊n/2⌋ − 1]

copy A[⌊n/2⌋..n − 1] to C[0.. ⌈n/2⌉ − 1]

Mergesort(B[0.. ⌊n/2⌋ − 1])

Mergesort(C[0.. ⌈n/2⌉ − 1])

Merge(*B*, *C*, *A*)

Cantidad de subproblemas

Costo de generar subproblemas (split) y componer resultados (join)

$$T(n) = 2T\left(\frac{n}{2}\right) + d(n),\ T(1) = 0$$

Tamaño de los subproblemas

¿Cómo esta definido Merge?

Ejemplo de Análisis de Tiempo de Ejecución

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$

//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ j \leftarrow 0;\ k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
    if $B[i] \leq C[j]$
        $A[k] \leftarrow B[i];\ i \leftarrow i+1$
    else $A[k] \leftarrow C[j];\ j \leftarrow j+1$
    $k \leftarrow k+1$
if $i = p$
    copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Las ecuaciones de recurrencia para este caso son, suponiendo que la operación de interés es la escritura en arreglos

$$T(n) = 2T\left(\frac{n}{2}\right) + 2n,\ T(1) = 0$$

¿Porqué 2n?

Ejemplo de Análisis de Tiempo de Ejecución

El análisis de soluciones Divide & Conquer recursivas demanda la resolución de ecuaciones de recurrencia. Consideremos el caso de MergeSort sobre un lenguaje imperativo:

ALGORITHM *Mergesort*(A[0..n − 1])
if n > 1

copy A[0..⌊n/2⌋ − 1] to B[0..⌊n/2⌋ − 1]
copy A[⌊n/2⌋..n − 1] to C[0..⌈n/2⌉ − 1]
} $\frac{n}{2} + \frac{n}{2} = n$ Escrituras en arreglo

Mergesort(B[0..⌊n/2⌋ − 1])

Mergesort(C[0..⌈n/2⌉ − 1])

Merge(*B*, *C*, *A*) } n Escrituras en arreglo

$$T(n) = 2T\left(\frac{n}{2}\right) + 2n,\ T(1) = 0$$

Ejemplo de Análisis de Tiempo de Ejecución

ALGORITHM *Merge*$(B[0..p-1],\ C[0..q-1],\ A[0..p+q-1])$

//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ \ j \leftarrow 0;\ \ k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
    if $B[i] \leq C[j]$
        $A[k] \leftarrow B[i];\ \ i \leftarrow i+1$
    else $A[k] \leftarrow C[j];\ \ j \leftarrow j+1$
    $k \leftarrow k+1$
if $i = p$
    copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

$$C_{merge}(p,q) = \sum_{i=0}^{p+q-1} 1 \quad = p+q-1+1 = p+q = n$$

Ejemplo de Análisis de Tiempo de Ejecución

ALGORITHM *Merge*$(B[0..p-1], C[0..q-1], A[0..p+q-1])$

//Merges two sorted arrays into one sorted array
//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted
//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C
$i \leftarrow 0;\ j \leftarrow 0;\ k \leftarrow 0$
while $i < p$ **and** $j < q$ **do**
    if $B[i] \leq C[j]$
        $A[k] \leftarrow B[i];\ i \leftarrow i+1$
    else $A[k] \leftarrow C[j];\ j \leftarrow j+1$
    $k \leftarrow k+1$
if $i = p$
    copy $C[j..q-1]$ to $A[k..p+q-1]$
else copy $B[i..p-1]$ to $A[k..p+q-1]$

Notar que si contamos comparaciones el análisis es distinto

Peor caso: ninguno de los dos arreglos se vacía antes de que el otro contenga solo un elemento (los elementos más pequeños van saliendo de manera alternada de ambos arreglos): $n-1$ comparaciones

Mejor caso: El **mejor caso** ocurre cuando uno de los dos arreglos "se agota" inmediatamente.
Ejemplo Si todos los elementos de B son **menores o iguales** a los de C

Ejemplo de Análisis de Tiempo de Ejecución

Realicemos en este caso el análisis mediante expansión de recurrencias:

$$\begin{aligned} T(n) &= 2 \times T(\tfrac{n}{2}) + 2 \times n \\ &= 2 \times (2 \times T(\tfrac{n}{4}) + 2 \times \tfrac{n}{2}) + 2 \times n \\ &= 4 \times T(\tfrac{n}{4}) + 2 \times n + 2 \times n \\ &= 4 \times (2 \times T(\tfrac{n}{8}) + 2 \times \tfrac{n}{4}) + 2 \times n + 2 \times n \\ &= 8 \times T(\tfrac{n}{8}) + 2 \times n + 2 \times n + 2 \times n \end{aligned}$$

Para resolver la ecuación conseguir un i tal que:

$$\frac{n}{2^i} = 1$$

La forma cerrada para T(n) es entonces:

$$T(n) = 2^i \times T(\frac{n}{2^i}) + i \times (2 \times n)$$

Ejemplo de Análisis de Tiempo de Ejecución

La forma cerrada para T(n) es entonces:

$$T(n) = 2^i \times T(\frac{n}{2^i}) + i \times (2 \times n)$$

Para resolver la ecuación conseguir un i tal que:

$$\frac{n}{2^i} = 1$$

$$\frac{n}{2^i} \Rightarrow n = 2^i \Rightarrow log_2\ n = i$$

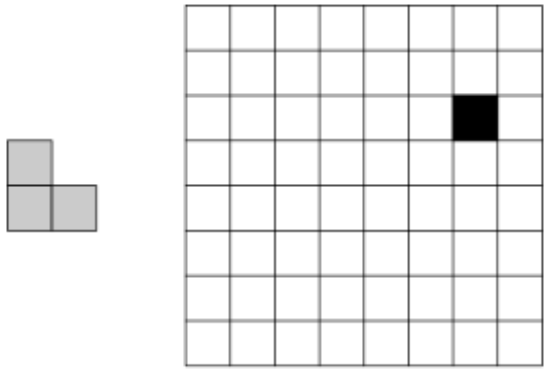
Reemplazando i por $log_2\ n$ tenemos:

$$\begin{aligned} T(n) &= 2^{\log_2 n} \times T(\frac{n}{2^{\log_2 n}}) + (\log_2 n) \times (2 \times n) \\ &= n \times T(1) + (\log_2 n) \times (2 \times n) \\ &= 0 + (\log_2 n) \times (2 \times n) \\ &= (\log_2 n) \times (2 \times n) \end{aligned}$$

Luego, la tasa de crecimiento de T es n $\log_2$ n. $\theta(nlog_2 n)$

Otro Ejemplo: El Problema de Trominós

Se desea cubrir una cuadrícula de $2^n x 2^n$ cuadros, con uno de ellos ya cubierto, usando trominós:



Pensemos en una solución Divide & Conquer para este problema



El Problema de Trominós: Análisis de Eficiencia



¿Tamaño de la entrada?

El Problema de Trominós: Análisis de Eficiencia

$$T(1) = 1,\ T(n) = 4T(n-1) + C_1$$

$$= 4(4T(n-2) + C_1) + C_1$$

$$= 16T(n-2) + 4C_1 + C_1$$

$$= 16(4T(n-3) + C_1) + 4C_1 + C_1$$

$$= 64T(n-3) + 16C_1 + 4C_1 + C_1$$

i = n - 1

La forma cerrada para T(n) es entonces:

$$= 4^i T(n-i) + \sum_{j=0}^{i-1} C_1 4^j \qquad = 4^i T(n-i) + C_1 \sum_{j=0}^{i-1} 4^j$$

El Problema de Trominós: Análisis de Ef

i = n - 1

La forma cerrada para T(n) es entonces:

$$= 4^i T(n-i) + C_1 \sum_{j=0}^{i-1} 4^j$$

$$= 4^{n-1} T(n-(n-1)) + C_1 \sum_{j=0}^{(n-1)-1} 4^j \qquad = 4^{n-1} T(1) + C_1 \sum_{j=0}^{n-2} 4^j$$

$$= 4^{n-1} + C_1 \sum_{j=0}^{n-2} 4^j \qquad = 4^{n-1} + C_1 \frac{4^{n-1}-1}{3}$$

$$= 4^{n-1} + C_1 \frac{4^{n-1}-1}{3} \qquad = \frac{3.4^{n-1} + C_1 4^{n-1} - C1}{3}$$

$$= \frac{3.4^{n-1} + C_1 4^{n-1} - C1}{3} \qquad = \frac{(3+C_1).4^{n-1} - C1}{3}$$

Geometric series:

$$\sum_{i=0}^{n} c^i = \frac{c^{n+1}-1}{c-1}$$

$$O(4^n)$$

De $O(4^n)$ a la Clase General $2^{O(n)}$

Expresamos la base de nuestro resultado asintótico en términos de base 2:

$4^n = (2^2)^n$ Aplicamos la ley de potencia de una potencia: $4^n = 2^{2n}$

Dado que el exponente es una función lineal, por definición de la notación Big O: $2n \in O(n)$

Sustituyendo el orden del exponente en la base: $O(4^n) \subseteq 2^{O(n)}$

De $O(4^n)$ a la Clase General $2^{O(n)}$

Cuando escribes $O(2^n)$, la "O" afecta a **toda la función**. Significa:

$$f(n) \leq c \cdot 2^n$$

En este caso: $f(n) \leq 2^{c \cdot n}$

Ejemplo: Ordenamiento

QuickSort:

caso base: secuencias de tamaño menor o igual a 1.

Solución: retornar la misma secuencia (ya está ordenada)

paso inductivo (para secuencias de tamaño mayor a 1):

Split: elegir un pivote, y separar la secuencia en dos partes, de manera tal que en la primera todos los elementos son menores o iguales que el pivote y en la segunda todos los elementos son mayores que el pivote

Split.xs = partition.xs

Join: Concatenar las dos secuencias (ya ordenadas)

Join.xs.ys = xs++ys

Ejemplo: Ordenamiento

```
QUICKSORT(A, p, r)
1  if p < r
2     then q ← PARTITION(A, p, r)
3          QUICKSORT(A, p, q)
4          QUICKSORT(A, q + 1, r)
```

```
PARTITION(A, p, r)
 1  x ← A[p]
 2  i ← p − 1
 3  j ← r + 1
 4  while TRUE
 5       do repeat j ← j − 1
 6            until A[j] ≤ x
 7          repeat i ← i + 1
 8            until A[i] ≥ x
 9          if i < j
10            then exchange A[i] ↔ A[j]
11            else return j
```

Ejemplo: Ordenamiento

```
PARTITION(A, p, r)
 1  x ← A[p]
 2  i ← p − 1
 3  j ← r + 1
 4  while TRUE
 5       do repeat j ← j − 1
 6            until A[j] ≤ x
 7          repeat i ← i + 1
 8            until A[i] ≥ x
 9          if i < j
10            then exchange A[i] ↔ A[j]
11            else return j
```

[2, 6, 4, 7, 9, 1, 5]

i j

x = 2

[2, 6, 4, 7, 9, 1, 5]

i j

[1, 6, 4, 7, 9, 2, 5]

i j

[1, 6, 4, 7, 9, 2, 5]

j i

<x | >=x

[1 | 6, 4, 7, 9, 2, 5]

Ejemplo: Ordenamiento

x = 5

```
PARTITION(A, p, r)
 1  x ← A[p]
 2  i ← p − 1
 3  j ← r + 1
 4  while TRUE
 5       do repeat j ← j − 1
 6            until A[j] ≤ x
 7          repeat i ← i + 1
 8            until A[i] ≥ x
 9          if i < j
10            then exchange A[i] ↔ A[j]
11            else return j
```

[5, 9, 7, 3, 1, 2, 8]

<x | >=x

[2, 1, 3, | 7, 9, 5, 8]

Ejemplo: Ordenamiento

```
QUICKSORT(A, p, r)
1  if p < r
2     then q ← PARTITION(A, p, r)
3          QUICKSORT(A, p, q)
4          QUICKSORT(A, q + 1, r)
```

Operación de. Interés: comparaciones

Best case

$$T(n) = 2T(\frac{n}{2}) + n,\ T(1) = 1$$

¿Y el worst case?

$$T(1) = 1$$
$$T(n) = c + T(n-1) + n$$

$\theta(n^2)$

TAREA

Ejemplo de Análisis de Tiempo de Ejecución

QuickSort, Best Case

Realicemos en este caso el análisis mediante expansión de recurrencias:

$$T(n) = 2T\left(\frac{n}{2}\right) + n = 2\left(2T\left(\frac{n}{4}\right) + \frac{n}{2}\right) + n = 4T\left(\frac{n}{4}\right) + 2n$$

$$= 4\left(2T\left(\frac{n}{8}\right) + \frac{n}{4}\right) + 2n \quad = 8T\left(\frac{n}{8}\right) + 3n$$

La forma cerrada para T(n) es entonces:

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$$

Para resolver la ecuación conseguir un i tal que:

$$\frac{n}{2^i} = 1$$

Ejemplo de Análisis de Tiempo de Ejecución

La forma cerrada para T(n) es entonces:

$$T(n) = 2^i T\left(\frac{n}{2^i}\right) + in$$

Para resolver la ecuación conseguir un i tal que:

$$\frac{n}{2^i} = 1$$

$$\frac{n}{2^i} \Rightarrow n = 2^i \Rightarrow log_2\ n = i$$

Reemplazando i por $log_2\ n$ tenemos:

$$T(n) = 2^{log_2 n} T\left(\frac{n}{2^{log_2 n}}\right) + log_2 n\ .\ n \quad = nT(1) + log_2 n\ .\ n$$

El más grande de los términos

$$= n + nlog_2 n$$

$$\theta(nlog_2 n)$$

Luego, la tasa de crecimiento de T es n $\log_2$ n.

Balance entre Split y Join

Los ejemplos anteriores de la estrategia Divide & Conquer nos muestran una relación que se da usualmente entre Split y Join

> En soluciones Divide & Conquer, en general sucede que si la división de problemas en subproblemas es “simple”, entonces el “pegado” de soluciones parciales para conseguir la solución global es “difícil”. Igualmente, si la división de problemas en subproblemas es “difícil”, el pegado es “simple”.

En los ejemplos, QuickSort es un algoritmo en el cual la división es difícil (y el pegado es fácil), mientras que MergeSort es un algoritmo en el cual la división es fácil (y el pegado difícil).

Otro Ejemplo...
Exponenciación

Problema: Computar a^n, donde n es un entero no negativo

Solución por Fuerza Bruta:

$$a^n = a * a * \ldots . * a \ (n \ veces)$$

¿Ecuación de recurrencia?

¿Cantidad de multiplicaciones?

Solución Divide & Conquer para computar a^n, con n >0 podemos computar

$$a^1 = a$$
$$a^n = a^{\lfloor \frac{n}{2} \rfloor} * a^{\lceil \frac{n}{2} \rceil}$$

¿Comparación con fuerza Bruta?

Análisis de Tiempo de Ejecución

Exponenciación

$$a^1 = a$$

$$a^n = a^{\lfloor \frac{n}{2} \rfloor} * a^{\lceil \frac{n}{2} \rceil}$$

Ecuación de recurrencia

$$T(1) = 0$$

$$T(n) = 2T\left(\frac{n}{2}\right) + 1$$

Operación de interés: multiplicaciones

Análisis de Tiempo de Ejecución

Exponenciación

$$T(1) = 0, \;\; T(n) = 2T(n/2) + 1$$

$$\begin{aligned} &= 2[2T(n/4) + 1] + 1 \\ &= 4T(n/4) + 2 + 1 \\ &= 4[2T(n/8) + 1] + 2 + 1 \\ &= 8T(n/8) + 4 + 2 + 1 \end{aligned}$$

La forma cerrada para T(n) es entonces:

$$= 2^i T\left(\frac{n}{2^i}\right) + 4 + 2 + 1 \quad = 2^i T\left(\frac{n}{2^i}\right) + \sum_{j=0}^{i-1} 2^j$$

Análisis de Tiempo de Ejecución

Exponenciación

$$= 2^i T\left(\frac{n}{2^i}\right) + 4 + 2 + 1 \quad = 2^i T\left(\frac{n}{2^i}\right) + \sum_{j=0}^{i-1} 2^j$$

$b^{log_b n} = n^{log_b b} = n$

Para resolver la ecuación conseguir un i tal que:

$$\frac{n}{2^i} = 1$$

$$sea\ i = log_2 n$$

$$= 2^{log_2 n} T\left(\frac{n}{2^{log_2 n}}\right) + \sum_{j=0}^{(log_2 n)-1} 2^j \quad = 2^{log_2 n} T(1) + \sum_{j=0}^{(log_2 n)-1} 2^j \quad = 2^{log_2 n} x\ 0 + \sum_{j=0}^{(log_2 n)-1} 2^j$$

$$= \sum_{j=0}^{(log_2 n)-1} 2^j \quad = \frac{2^{log_2 n} - 1}{2 - 1} \quad = 2^{log_2 n} - 1 = n - 1$$

Geometric series:

$$\sum_{i=0}^{n} c^i = \frac{c^{n+1} - 1}{c - 1}$$

Análisis de Tiempo de Ejecución

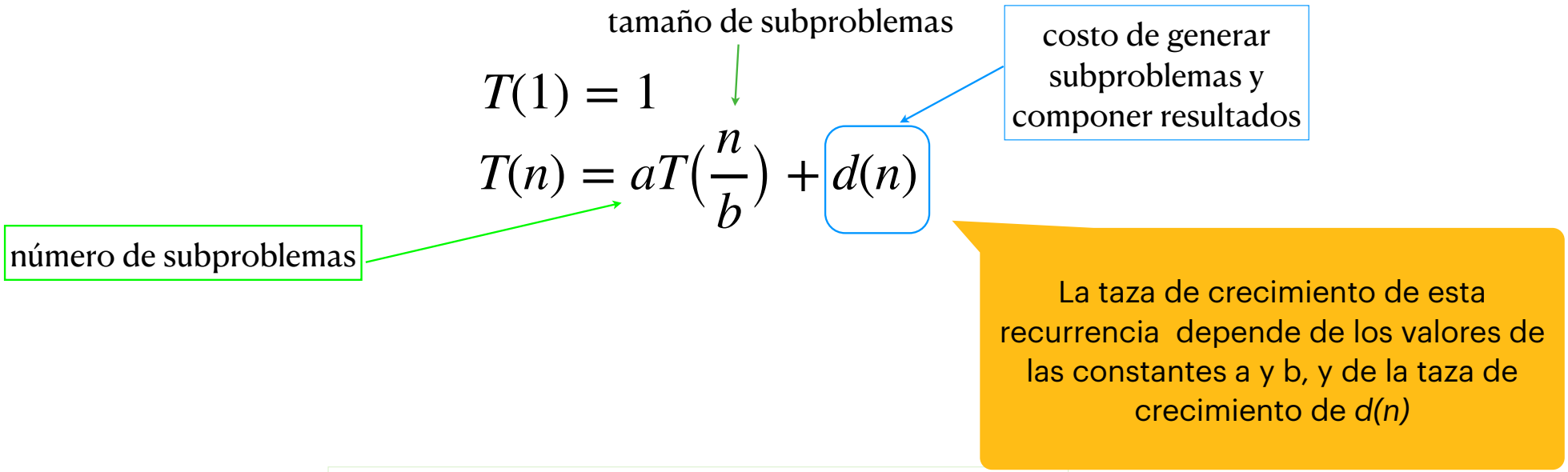
Exponenciación

$$\Theta(n)$$

Al igual que la solución fuerza Bruta, realiza n-1 multiplicaciones!

Tiempo de Ejecución de Algoritmos Divide & Conquer

Dijimos que el análisis de tiempo de ejecución de algoritmos recursivos requiere la solución de ecuaciones de recurrencias. En una cantidad importante de casos, estas ecuaciones de recurrencias suelen tener la siguiente forma:



Recurrencia Divide & Conquer general

Tiempo de Ejecución de Algoritmos Divide & Conquer/Decrease & Conquer

Teorema Maestro

IMPORTANTE

Deben satisfacerse las siguientes condiciones para su aplicación

$a \geq 1 \quad$ y $\quad b > 1$

$Si\ d(n) = 0\ entonces\ T(n) = \Theta(n^{log_b a})$

$Si\ d(n) \in \Theta(n^{\alpha})\ donde\ \alpha \geq 0\ entonces$

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Resultados análogos se mantienen para O y Ω

Ejemplo de Análisis de Tiempo de Ejecución

Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Consideremos ahora el caso de MergeSort

Las ecuaciones de recurrencia resultantes son las siguientes:

$$T(n) = 2T\left(\frac{n}{2}\right) + 2n,\ T(1) = 0$$

Tenemos entonces: $a = 2, b = 2, d(n) = 2 \times n, d(n) \in \Theta(n)$ $\alpha = 1$

$a = b^{\alpha}\ (2 = 2^1)$ Entonces $T(n) \in \Theta(n^{\alpha} \log_b n)$

$$T(n) \in \Theta(n \log_2 n)$$

Análisis de Tiempo de Ejecución
Teorema Maestro

Observar que el Teorema Maestro nos permite encontrar la **clase de eficiencia de la solución sin resolver la recurrencia**. Pero, este enfoque solo puede establecer el orden de crecimiento de la solución dentro de una constante multiplicativa desconocida, mientras que resolver una ecuación de recurrencia con una condición inicial específica proporciona **una respuesta exacta** (al menos para valores de n que sean potencias de b)

Ejemplo de Análisis de Tiempo de Ejecución
Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Consideremos la ecuación:

$$T(1) = 1; T(n) = 4T\left(\frac{n}{2}\right) + n$$

Tenemos entonces: $a = 4,\ b = 2,\ d(n) = n,\ d(n) \in \Theta(n) \qquad \alpha = 1$

$a > b^{\alpha}\ (4 > 2^1)$ entonces $T(n) \in \Theta(n^{log_b a})$

$$T(n) \in \Theta(n^{log_2 4})$$

$$T(n) \in \Theta(n^2)$$

Ejemplo de Análisis de Tiempo de Ejecución

Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Consideremos la ecuación:

$$T(1) = 1; T(n) = T\left(\frac{n}{2}\right) + 1$$

Tenemos entonces: $a = 1,\ b = 2,\ d(n) = 1,\ \alpha = 0$

$a = b^{\alpha}\ (1 = 2^0)$ entonces $T(n) \in \Theta(n^{\alpha} log_b n)$

$T(n) \in \Theta(n^0 log_2 n)$

$T(n) \in \Theta(log\ n)$

Ejemplo de Análisis de Tiempo de Ejecución
Teorema Maestro

Consideremos las ecuaciones:

$$T(1) = 1; T(n) = 4T(n-1) + 1$$

$$a = 1,\ b = 1,\ d(n) = 1,\ \alpha = 0 \qquad b > 1$$

No es posible aplicar el Teorema Maestro

$$T(1) = 1; T(n) = T\left(\frac{n}{2}\right) + 2^n$$

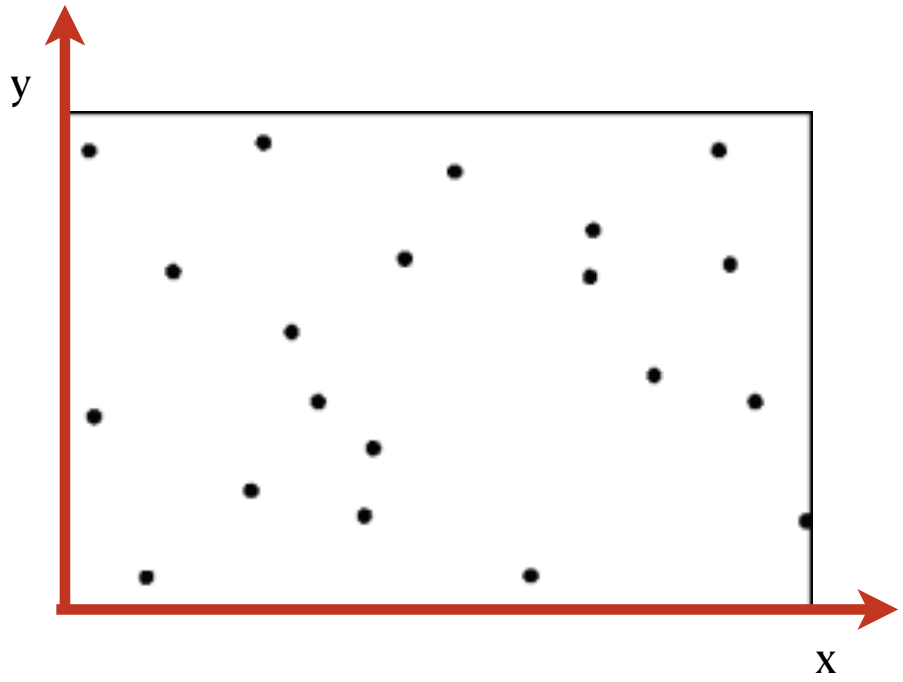
$$a = 1,\ b = 2,\ d(n) = 2^n,\ \alpha = ?$$

No es posible aplicar el Teorema Maestro

$$d(n) \notin \Theta(n^\alpha)$$
$$d(n) \notin O(n^\alpha)$$

Otro Ejemplo: Puntos Más Cercanos entre Sí

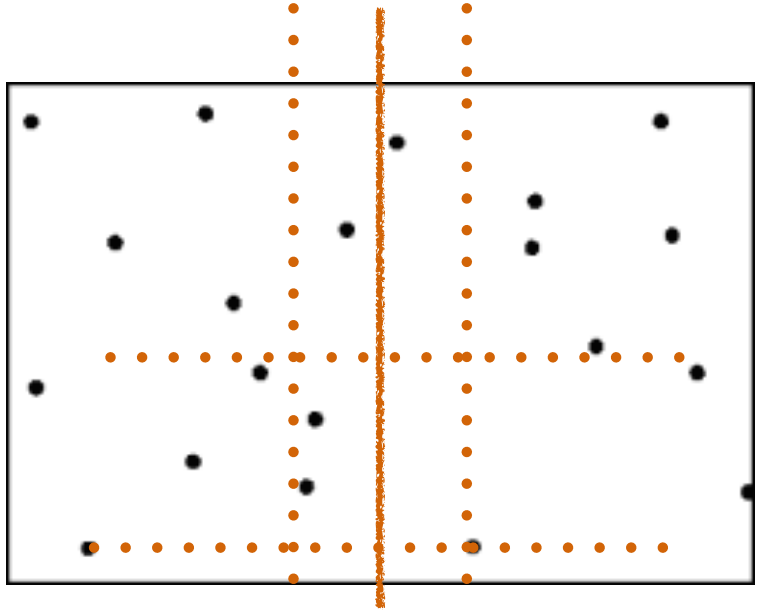
Dado un conjunto de puntos en el plano, se desea encontrar el par de puntos distintos cuya distancia entre ellos es la menor:



Pensemos en una solución Divide & Conquer para este problema.

Otro Ejemplo: Puntos Más Cercanos entre Sí

Dado un conjunto de puntos en el plano, se desea encontrar el par de puntos distintos cuya distancia entre ellos es la menor:



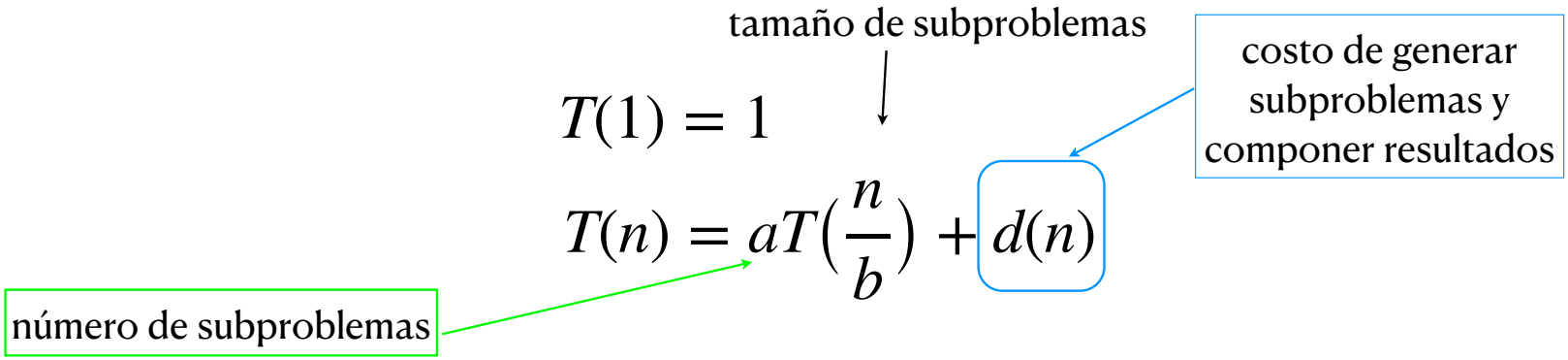
Pensemos en una solución Divide & Conquer para este problema.

Puntos Más Cercanos entre Sí
Ecuación de Recurrencia

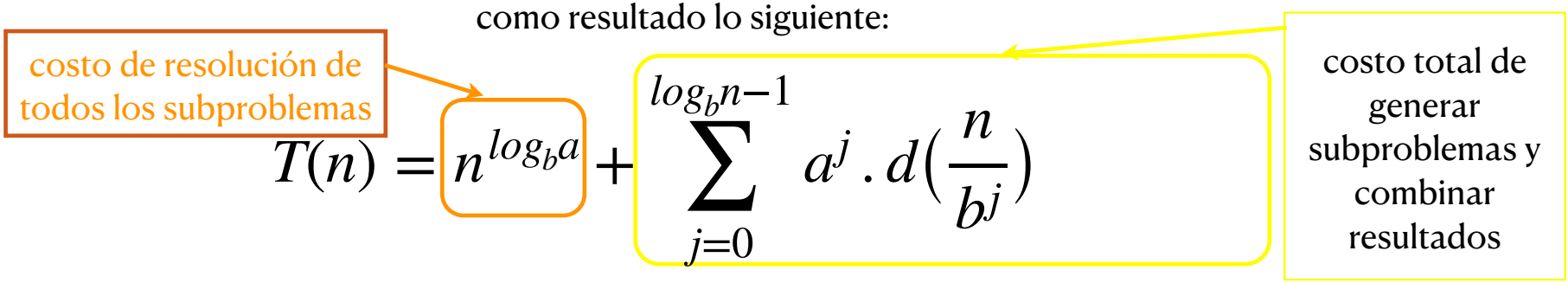
$$T(n) = 2T(\frac{n}{2}) + d(n)$$

Qué tiempo debe tener d(n) para lograr un tiempo n.log n?

Teorema Maestro

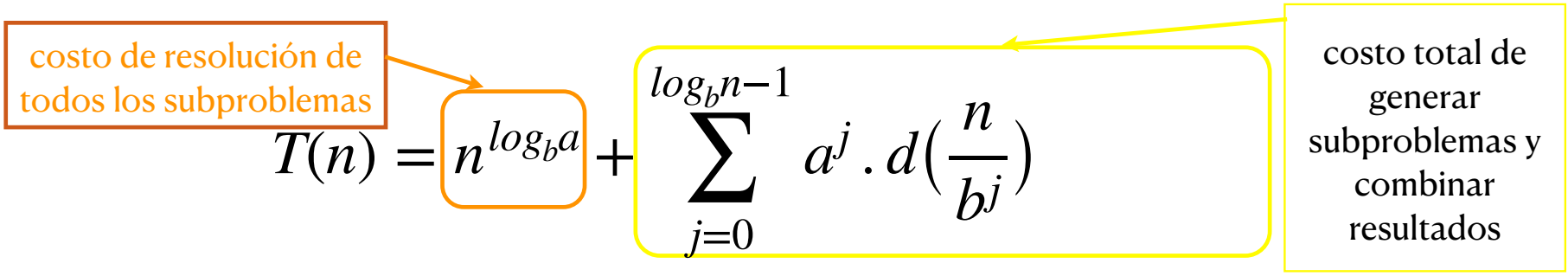


Podemos resolverlas de forma genérica. La resolución de estas ecuaciones de recurrencia da como resultado lo siguiente:



Teorema Maestro

$$T(n) = aT\left(\frac{n}{b}\right) + d(n)$$



$$Si\ d(n) = 0 \implies T(n) = n^{log_b a}$$

$$entonces\ T(n) = \Theta(n^{log_b a})$$

Teorema Maestro

costo de resolución de todos los subproblemas

$$T(n) = n^{log_b a} + \sum_{j=0}^{log_b n - 1} a^j \cdot d\left(\frac{n}{b^j}\right)$$

costo total de generar subproblemas y combinar resultados

Si $d(n) \in \Theta(n^\alpha)$ *donde* $\alpha \geq 0$ *entonces*

$$T(n) \in \begin{cases} \Theta(n^\alpha) & \text{si } a < b^\alpha, \\ \Theta(n^\alpha \log_b n) & \text{si } a = b^\alpha, \\ \Theta(n^{\log_b a}) & \text{si } a > b^\alpha . \end{cases}$$

Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

costo de resolución de todos los subproblemas

$$T(n) = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j \cdot d\left(\frac{n}{b^j}\right)$$

costo total de generar subproblemas y combinar resultados

Si $d(n) \in \Theta(n^{\alpha})$ *donde* $\alpha \geq 0$ *entonces*

$$T(n) = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j \cdot \left(\frac{n}{b^j}\right)^{\alpha} = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j \cdot \left(\frac{n^{\alpha}}{b^{j\alpha}}\right) = n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} n^{\alpha} \cdot \left(\frac{a}{b^{\alpha}}\right)^j$$

$$= n^{\log_b a} + n^{\alpha} \cdot \sum_{j=0}^{\log_b n - 1} \left(\frac{a}{b^{\alpha}}\right)^j$$

Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Si $d(n) \in \Theta(n^{\alpha})$ *donde* $\alpha \geq 0$ *entonces* $T(n) = n^{log_b a} + n^{\alpha} . \sum_{j=0}^{log_b n - 1} \left(\frac{a}{b^{\alpha}}\right)^j$

si $a = b^{\alpha}$ *entonces* $T(n) = n^{log_b a} + n^{\alpha} . \sum_{j=0}^{log_b n - 1} 1^j = n^{log_b a} + n^{\alpha} . \sum_{j=0}^{log_b n - 1} 1 = n^{log_b a} + n^{\alpha} . log_b n - 1 + 1$

$= n^{log_b a} + n^{\alpha} . log_b n \quad = n^{log_b (b^{\alpha})} + n^{\alpha} . log_b n \quad = n^{\alpha . log_b b} + n^{\alpha} . log_b n \quad = n^{\alpha} + n^{\alpha} . log_b n$

$$\Theta(n^{\alpha} \log_b n)$$

Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Si $d(n) \in \Theta(n^{\alpha})$ *donde* $\alpha \geq 0$ *entonces* $T(n) = n^{log_b a} + n^{\alpha} \cdot \sum_{j=0}^{log_b n - 1} \left(\frac{a}{b^{\alpha}}\right)^j$

si $a < b^{\alpha}$ *entonces* $log_b a < log_b(b^{\alpha})$ $log_b a < \alpha$

Monotonía del logaritmo

$\Theta(n^{\alpha})$

$$T(n) = n^{log_b a} + n^{\alpha} \cdot \underbrace{\sum_{j=0}^{log_b n - 1} \left(\frac{a}{b^{\alpha}}\right)^j}_{< 1}$$

La serie converge a una constante $\frac{1}{1 - \frac{a}{b^{\alpha}}}$

Teorema Maestro

$$T(n) \in \begin{cases} \Theta(n^{\alpha}) & \text{si } a < b^{\alpha}, \\ \Theta(n^{\alpha} \log_b n) & \text{si } a = b^{\alpha}, \\ \Theta(n^{\log_b a}) & \text{si } a > b^{\alpha}. \end{cases}$$

Si $d(n) \in \Theta(n^{\alpha})$ *donde* $\alpha \geq 0$ *entonces* $\quad T(n) = n^{log_b a} + n^{\alpha} . \sum_{j=0}^{log_b n - 1} \left(\frac{a}{b^{\alpha}}\right)^j$

> 1

si $a > b^{\alpha}$ *entonces* $\quad T(n) = n^{log_b a} + n^{\alpha} . \underbrace{\sum_{j=0}^{log_b n - 1} \left(\frac{a}{b^{\alpha}}\right)^j}$

domina el último término de la serie $\left(\frac{a}{b^{\alpha}}\right)^{log_b n} = \frac{a^{\log_b n}}{\left(b^{\alpha}\right)^{\log_b n}} = \frac{a^{\log_b n}}{b^{\alpha \log_b n}} = \frac{a^{\log_b n}}{n^{\alpha \log_b b}}$

$$T(n) = n^{log_b a} + n^{\alpha} . \left(\frac{n^{log_b a}}{n^{\alpha}}\right) \qquad T(n) = 2n^{log_b a}$$

$$\Theta(n^{log_b a})$$